ONSURITY — ENGINEERING — BENGALURU

Software Development Engineer I, Java

Your first engineering job, at a company small enough that what you write this week is live next week. You'll be taught, reviewed and stretched — and trusted with real work far earlier than most places would. **The floor is the same at every level here. What changes is how much of it you carry alone.**

LEVEL	TRACK	EXPERIENCE	LOCATION	STACK
SDE I	Individual contributor	0–2 years	Bengaluru	Java · Node · React

ABOUT US

Healthcare benefits shouldn't be a luxury

Onsurity is a fast-growing employee healthcare benefits platform. We give SMEs, start-ups and enterprises flexible healthcare subscriptions — discounted medicine, health checkups, fitness plans, free doctor teleconsultations and insurance, in one membership. Our plans aren't limited to full-time employees; we also cover contractual workers, interns, freelancers and consultants, because inclusion is the whole point.

We've raised a **$45M Series B led by Creaegis**, count cricketing legend **Anil Kumble** as a strategic advisor and investor, featured in season one of Disney+ Hotstar's *The Great Indian Disruptors*, and were named a **Dun & Bradstreet Start-up 50 Trailblazer**. More at onsurity.com — or find us on LinkedIn, Instagram and YouTube.

THE ROLE

What this job actually is

We're hiring engineers at the start of their careers who we think will be very good in three years, and we're willing to invest to get there.

You'll join a team that owns a domain — claims, wallet, subscriptions, provider network or payments — and you'll take features inside it. Not toy tickets kept away from anything important. Real features, with a senior engineer reviewing your design before you build and your code before it ships.

Java and Spring Boot are where you'll spend most of your time. You'll also touch React screens and Node services, because features here don't stop at a language boundary. You're not expected to know all of it on day one. You are expected to learn it faster than is comfortable.

You'll be given context rather than instructions: why the feature exists, who it's for, what happens if it's wrong. Insurance is a domain with real rules, and a member may be relying on your screen during a hospital visit. Understanding that is part of the job, not background noise.

The deal is simple. We'll teach you, review you honestly, and let you near things that matter early. In return we expect you to ask a lot of questions, take feedback without flinching, and get measurably better every month.

THE LEVEL

What SDE I means here

You're not expected to arrive knowing the answers. These six are what the level does ask for.

SCOPE	A feature, or a well-defined piece of one. Someone helps you shape it.
AMBIGUITY	The problem arrives mostly specified. You notice the parts that aren't, and ask.
INFLUENCE	Your own work, and the quality of every file you touch on the way past.
JUDGEMENT	Knowing what you can decide and what you should ask about. Both are skills.
CONSEQUENCE	You test what you write, you don't break the build, and you say so the moment you do.
SLOPE	You need less help this month than last month. That's the whole job.

YOUR WORK

What you'll do

DESIGN

- Write the **LLD** for what you build — the schema, the API contract, the states a record can be in — and walk someone senior through it before you write the code.
- Sit in the **HLD** discussions for your domain and follow them properly. Ask when you lose the thread; at this level not following is a bigger problem than not knowing.
- Say out loud what you think the failure cases are. Being wrong about them in a review costs nothing. Missing them in production costs a member.

BUILD

- Write Java services and React screens for a domain you're still learning, against a design you've walked through with someone senior first.
- Turn a ticket into working, tested code — and say early when the ticket itself is wrong. Spotting that is worth more than finishing it quietly.
- Write the tests as you go. Not as a follow-up ticket, and not because someone asked.
- Read a great deal more code than you write for the first few months. That's the fastest way in.

RUN

- Stay with what you shipped. Check the dashboard after a release rather than moving straight to the next thing.
- Debug your own bugs first — logs, then the code, then a person. Come with what you already ruled out.
- Shadow the on-call rotation before you join it, so the first real incident isn't your first look at the tooling.
- Treat member data as confidential by default. Health and money information isn't something you get a second chance with.

GROW

- Take code review as the fastest free teaching you'll ever get. Argue when you disagree; change your mind when you're wrong.
- Ask early and in the open. A blocked afternoon is expensive; a question costs nothing.
- Learn the domain, not only the syntax – what a claim is, why a wallet balance moves, what a regulator requires.
- Aim to be the person someone else asks by the end of your first year.

THE BAR

NON-NEGOTIABLE

How we expect you to operate

We'll teach you the technical parts. These five aren't things we can teach, so we select for them.

High ownership

Your feature is yours until it's live and working, not until the pull request is merged. If it breaks the next week, you want to be the one who finds out first.

Highly motivated and self-driven

You look for the next useful thing rather than waiting to be assigned it, and you say clearly and early when something is going to slip.

Extensive AI use

You already use Copilot, Claude or ChatGPT daily – and you can still explain every line you ship, because the point is to learn faster, not to skip learning.

Fast, with a floor under it

You ship in days, not weeks, and you don't buy that speed by skipping tests or leaving a mess behind you.

Direct and low-ego

You say "I don't know" quickly, you ask the question you're slightly embarrassed to ask, and you take a hard code review as help rather than as criticism.

AI

How we expect you to use AI

AI-assisted development is the default here, not a novelty. For someone early in their career it's a genuine accelerant – and the easiest way to stall.

- Use Copilot, Claude or ChatGPT every day – scaffolding, tests, refactors, debugging, understanding an unfamiliar codebase.
- Read and own every line you ship. "The AI wrote it" is not an explanation for a production incident.
- **If you can't explain it in review, don't ship it.** This is the one we'll hold you to hardest at this level.

- Use it to learn faster, not to skip learning. Ask it *why*, make it show you the alternative, then decide for yourself.
- Be careful where it's weakest — our domain rules, security-sensitive paths, and anything touching money or member health data. Verify those yourself.

MINIMUM

Minimum qualifications

Kafka, monitoring and observability, and both altitudes of system design are floor requirements at every level here, including this one. What differs at SDE I is the depth we expect, not whether you have it at all.

- **0–2 years** of professional experience, internships included. New graduates are welcome and expected.
- **Solid Java** — you can write clean, correct code without the IDE doing the thinking for you.
- **Data structures and algorithms** you genuinely understand, rather than patterns you've memorised.
- **SQL** — you can write a join, and reason about why a query is slow.
- **CS fundamentals** — OOP, the basics of concurrency, HTTP, and how a database and a network actually behave.
- **LLD** — you can design the schema and the API contract for a small feature, and name its edge cases without being walked through them.
- **HLD** — you can draw a system's components, say what calls what, and point at where it would break. A real attempt, not a finished architecture.
- **Kafka** or a similar event-driven system — you've built with it, and you can explain what a partition, a consumer group and consumer lag actually are.
- **Monitoring and observability** — you answer questions with dashboards, logs and traces rather than guesses, and you've added a metric or an alert yourself.
- **Git**, and the habit of small, readable commits someone else can follow.
- **Some Spring Boot** exposure, or clear evidence you get productive in a new framework quickly.
- **Daily AI use** already part of how you work.
- **Clear communication** — you can explain what you built, and say precisely what you're stuck on.
- **Bachelor's degree** in Computer Science, Engineering or a related field, or equivalent practical evidence.

PREFERRED

Preferred qualifications

- An internship, or a project where people other than you actually used what you built.
- Anything you shipped end to end — small is fine, a side project is fine — that you can explain the trade-offs on.
- React, or any modern frontend framework.
- Node.js or TypeScript.
- Competitive programming, open-source contributions, or a portfolio we can read.
- You've built something with an LLM API, not only used AI tooling.
- Exposure to cloud, Docker or CI — even at a hobby level.
- Any exposure to healthcare, insurance or fintech.

OUR STACK

What you'd work with

You won't know all of this on day one, and we don't expect you to. You should expect to have touched most of it by the end of your first year.

language	Java 17+, Node.js, TypeScript
backend	Spring Boot, REST APIs, JUnit
frontend	React web, React Native mobile app
data	MySQL, PostgreSQL, MongoDB, Redis
streaming	Kafka
platform	AWS, Docker, Kubernetes, GitHub Actions
observability	Prometheus, Grafana, Sentry, ELK
ai	Copilot, Claude, ChatGPT — in the daily workflow

THE BAR, TESTED

What we'll assess

Six dimensions. At this level we're reading fundamentals, design instinct and rate of learning — not the length of your CV.

Coding and fundamentals

Correct, readable code under time pressure, plus data structures, complexity and enough of how databases and networks behave to reason about a real system.

Low-level design

An API and a data model for a small feature. Clear contracts, sensible tables, and the edge cases you raise before we do.

High-level design

A small system sketched out loud — components, what calls what, where it breaks. We're reading instinct and questions, not a finished architecture.

Debugging

Something that doesn't work. What you check first, how you narrow it down, and whether you guess or verify.

Project deep-dive

Something you built. Why those choices, what you'd do differently, and how much of it you can still explain.

Learning and collaboration

How you handle a concept you haven't met, feedback you disagree with, and admitting you don't know.

READ TWICE

This role is probably not for you if

- You want months of structured training before touching anything real. You'll be in production code in your first weeks.
- You expect design to be someone else's job until you're senior. You'll be asked for a schema and a failure mode in your first months, and in the interview before that.
- You want to write code and never read anyone else's. Most of the learning here comes the other way round.
- You'd rather stay quiet than say you're stuck. Silence is the one failure mode we can't work with.
- You want to specialise immediately. The first year is deliberately broad — backend, a bit of frontend, some infrastructure.
- You're uncomfortable working where a bug can delay someone's hospital admission. That's the seriousness this team runs at.

EQUAL OPPORTUNITY

Onsurity is an equal-opportunity employer. We hire on capability and evidence, and we're committed to an inclusive workplace — the same philosophy behind healthcare plans that cover the people other plans leave out.